МИНОБРНАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении лабораторной работы №1 по дисциплине «Теория алгоритмов»

Студент группы 5130201/40002 _____ Гасов А.Е.

Преподаватель _____ Востров А.В.

«___» _____ 2026 г.

Санкт-Петербург
2026

Содержание

Введение **3**

1 Постановка задачи **4**

2 Математическое описание **5**
2.1 Конечный автомат 5
2.2 Клеточный автомат 5
2.3 Окрестность фон Неймана 5
2.4 Тороидальные граничные условия 6
2.5 Правило перехода 6
2.6 Классы Вольфрама 7

3 Особенности реализации **8**
3.1 Класс Neumann_class 8
3.2 Класс GridWidget 9
3.3 Класс MainWindow 9

4 Результаты работы **13**
4.1 Работа программы на случайных начальных условиях 13
4.2 Работа программы на ручных начальных условиях 13

5 Анализ клеточного автомата **15**
5.1 Паттерны 15
5.1.1 Тривиальные начальные условия 15
5.1.2 Шахматные доски 15
5.1.3 Диагональные волны (дрейф вправо) 15
5.2 Классификация по Вольфраму 16

Заключение **17**

Список литературы **18**

Введение

В лабораторной работе был реализован двумерный клеточный автомат с окрестностью фон Неймана на поле с тороидальными границами. Правило перехода было задано номером варианта. Программа позволяет выбрать ширину поля, число итераций и начальную конфигурацию (случайную или заданную вручную). Также был проведен анализ поведения клеточного автомата - изменение плотности, получившиеся паттерны, класс по Вольфраму.

1 Постановка задачи

Реализовать двумерный клеточный автомат с окрестностью фон Неймана в соответствии с полученным номером N. Граничные условия - тороидальные. Пользователь определяет ширину поля и количество итераций. Необходимо учесть возможность ввода различных начальных условий (как вручную, так и случайным образом) по выбору пользователя.

Номер вычисляется по формуле

$$N = \text{номер варианта} \cdot 11 \cdot \text{год рождения} \cdot \text{день} \cdot \text{месяц}.$$

Для номера варианта $= 9$: $N = 9 \cdot 11 \cdot 2006 \cdot 21 \cdot 12 = 50045688$.

2 Математическое описание

2.1 Конечный автомат

Конечным автоматом называется набор

$$A = (S, \Sigma, Y, s_0, \delta, \lambda),$$

где

- S - конечное множество состояний
- Σ - конечное множество входных сигналов
- Y - конечное множество выходных сигналов
- $s_0 \in S$ - начальное состояние
- $\delta : S \times \Sigma \to S$ - функция переходов
- $\lambda : S \times \Sigma \to Y$ - функция выходов

2.2 Клеточный автомат

Клеточный автомат - это регулярная решетка одинаковых конечных автоматов (клеток), каждый из которых в момент времени t находится в одном из конечных состояний. Состояние клетки на следующем шаге вычисляется по текущему состоянию клетки и состояниям клеток ее окрестности, причем правило одно и то же для всех клеток. Обновление всех клеток решетки происходит одновременно при переходе с шага t на t+1.

Каждая клетка - автомат $A = (S, \Sigma, Y, s_0, \delta, \lambda)$, где

- $S = \{0, 1\}$ - состояния клетки (0 - мертвая, 1 - живая)
- $\Sigma = \{0, 1\}^5$ - входной сигнал (набор состояний клеток окрестности фон Неймана и самой клетки)
- $Y = \{0, 1\}$ - выходной сигнал клетки, ее состояние на шаге $t + 1$
- $\delta : S \times \Sigma \to S$ - функция переходов, булева функция пяти переменных $f(S_T, S_L, S_C, S_R, S_B)$
- $\lambda : S \times \Sigma \to Y$ - функция выходов; выход клетки равен её новому состоянию, поэтому $\lambda = \delta$
- s_0 - начальное состояние клетки

2.3 Окрестность фон Неймана

Окрестность фон Неймана клетки - это совокупность клеток, имеющих с ней общую сторону. Пример окрестности фон Неймана представлен на Рис.1.

	S_T	
S_L	S_C	S_R
	S_B	

Рис. 1: Окрестность фон Неймана: верх (S_T), лево (S_L), центр (S_C), право (S_R), низ (S_B)

2.4 Тороидальные граничные условия

Рассматриваются тороидальные граничные условия: поле ширины n замыкается в тор, а индексы клеток берутся по модулю n. Тогда соседи клетки $S_{i,j} = S_C$ определяются как

$$S_T = S_{(i-1) \bmod n,\, j}, \quad S_B = S_{(i+1) \bmod n,\, j}, \quad S_L = S_{i,\,(j-1) \bmod n}, \quad S_R = S_{i,\,(j+1) \bmod n}.$$

2.5 Правило перехода

Состояние клетки на шаге $t+1$ вычисляется по состояниям клеток окрестности на шаге t:

$$S_C^{t+1} = f\big(S_T^t,\, S_L^t,\, S_C^t,\, S_R^t,\, S_B^t\big).$$

Окрестность из пяти клеток дает $2^5 = 32$ возможных конфигураций, а функций $f : \{0,1\}^5 \to \{0,1\}$ существует 2^{32}. Поэтому правило однозначно кодируется 32-битным числом N: его двоичная запись, дополненная нулями слева до 32 разрядов, является столбцом значений функции f в таблице истинности, строки которой упорядочены по возрастанию двоичного числа $S_T S_L S_C S_R S_B$ (старший бит - верхний сосед, младший - нижний).

Номер варианта $N = 50045688$ в двоичной записи равен

$$N = 00000010111110111010001011111000_2,$$

Таблица истинности приведена в таблице 1.

i	S_T	S_L	S_C	S_R	S_B	f
0	0	0	0	0	0	0
1	0	0	0	0	1	0
2	0	0	0	1	0	0
3	0	0	0	1	1	0
4	0	0	1	0	0	0
5	0	0	1	0	1	0
6	0	0	1	1	0	1
7	0	0	1	1	1	0
8	0	1	0	0	0	1
9	0	1	0	0	1	1
10	0	1	0	1	0	1
11	0	1	0	1	1	1
12	0	1	1	0	0	1
13	0	1	1	0	1	0
14	0	1	1	1	0	1
15	0	1	1	1	1	1
16	1	0	0	0	0	1
17	1	0	0	0	1	0
18	1	0	0	1	0	1
19	1	0	0	1	1	0
20	1	0	1	0	0	0
21	1	0	1	0	1	0
22	1	0	1	1	0	1
23	1	0	1	1	1	0
24	1	1	0	0	0	1
25	1	1	0	0	1	1
26	1	1	0	1	0	1
27	1	1	0	1	1	1
28	1	1	1	0	0	1
29	1	1	1	0	1	0
30	1	1	1	1	0	0
31	1	1	1	1	1	0

Таблица 1: Таблица истинности функции f для $N = 50045688$

2.6 Классы Вольфрама

Классы Вольфрама - классификация клеточных автоматов, основанная на их поведении. Вольфрам выделил 4 класса:

- Класс I. После некоторого количества шагов система стабилизируется: все клетки поля переходят в одинаковое состояние.
- Класс II. Состояния могут быть различными, однако клетки образуют наборы статичных или периодических простых структур.
- Класс III. Образуются сложные структуры, характер взаимодействия которых во многих случаях выглядит хаотическим.
- Класс IV. Сложные структуры; очень сложное, порой непредсказуемое (хаотичное) взаимодействие.

3 Особенности реализации

Программа написана на C++ с использованием Qt6 для реализации графической состовляющей. Программа состоит из 3 классов:

- **Neumann_class** - хранит номер варианта и по нему строит таблицу истинности
- **GridWidget** - виджет поля-решетки
- **MainWindow** - главное окно программы, на котором задаются начальные условия, размеры поля, а также вычисляется состояние поля на i-ой итерации

3.1 Класс Neumann_class

Функция fill_f_col

Вход: `int variant` - номер варианта

Выход: `std::vector<int> f_col` - 32 бита номера варианта (столбец значений таблицы истинности)

Описание: раскладывает число на двоичные разряды, заполняя вектор с конца, так что `f_col[0]` - старший бит, а `f_col[31]` - младший. Недостающие старшие разряды остаются нулями.

```
1  void Neumann_class::fill_f_col(){
2      f_col.assign(32, 0);
3      int var_copy = variant;
4      int i = 31;
5      while(var_copy){
6          f_col[i] = var_copy % 2;
7          var_copy /= 2;
8          i--;
9      }
10 }
```

Листинг 1: fill_f_col

Функция fill_neumann

Вход: `std::vector<int> f_col` - 32 бита номера варианта

Выход: `std::map<std::string, int> neumann` - отображение, задающее правило работы автомата (таблица истинности)

Описание: для каждого $i = 0..31$ формирует ключ - строку из пяти битов числа i, начиная со старшего (порядок: верх, лево, центр, право, низ), - и сопоставляет ему значение `f_col[i]`.

```
void Neumann_class::fill_neumann(){
    for(int i = 0; i < 32; i++){
        std::string str_i = "";
        for(int bias = 0; bias < 5; bias++){
            str_i += std::to_string( (i >> (4 - bias)) & 1 );
        }
        neumann[str_i] = f_col[i];
    }
}
```

Листинг 2: fill_neumann

3.2 Класс GridWidget

Функция mousePressEvent

Вход: `QMouseEvent* event` - событие нажатия кнопки мыши

Выход: сигнал `cell_clicked(row, col)`

Описание: размер клетки получается делением ширины и высоты виджета на число столбцов и строк. Координаты клика делятся на размер клетки, получаются индексы `row` и `col`. Если они лежат внутри поля, испускается сигнал `cell_clicked`.

```
void GridWidget::mousePressEvent(QMouseEvent* event){
    if (grid.empty()) return;
    int rows = static_cast<int>(grid.size());
    int cols = static_cast<int>(grid[0].size());
    double cell_w = width() / static_cast<double>(cols);
    double cell_h = height() / static_cast<double>(rows);

    int col = static_cast<int>(event->position().x() / cell_w);
    int row = static_cast<int>(event->position().y() / cell_h);
    if (row >= 0 && row < rows && col >= 0 && col < cols){
        emit cell_clicked(row, col);
    }
}
```

Листинг 3: mousePressEvent

3.3 Класс MainWindow

Функция on_init

Вход: значения полей ширины и числа итераций, состояние переключателя начальных условий

Выход: новое поле `cells`, передаваемое в `GridWidget`

Описание: слот кнопки «Перегенерировать» (вызывается также при запуске программы). Считывает ширину поля и число итераций, обнуляет счетчик итераций и создает матрицу из нулей. Если выбран режим «Случайные»,

каждой клетке присваивается 0 или 1 с вероятностью 1/2; в режиме «Вручную» поле остается пустым.

```
void MainWindow::on_init(){
    field_size = width_spin->value();
    max_iterations = iters_spin->value();
    iteration = 0;

    cells.assign(field_size, std::vector<int>(field_size, 0));
    if (random_radio->isChecked()){
        for (int r = 0; r < field_size; ++r){
            for (int c = 0; c < field_size; ++c){
                cells[r][c] = std::rand() % 2;
            }
        }
    }
    grid_widget->set_grid(cells);
}
```

Листинг 4: on_init

Функция on_cell_clicked

Вход: `int row, int col` - индексы клетки, по которой кликнули

Выход: изменённое поле `cells`

Описание: слот сигнала `cell_clicked`. Если выбран режим «Вручную», значение клетки инвертируется и поле передаётся виджету на перерисовку; в режиме «Случайные» клик игнорируется.

```
void MainWindow::on_cell_clicked(int row, int col){
    if (manual_radio->isChecked() == false) return;
    cells[row][col] = cells[row][col] ? 0 : 1;
    grid_widget->set_grid(cells);
}
```

Листинг 5: on_cell_clicked

Функция torus_index

Вход: `int i` - индекс строки или столбца, $i \in [-1, n]$

Выход: `int` - индекс из $[0, n-1]$

Описание: реализует тороидальные границы: $-1 \mapsto n-1$, $n \mapsto 0$, остальные индексы не меняются.

```
int MainWindow::torus_index(int i){
    if (i < 0) return i + field_size;
    if (i >= field_size) return i - field_size;
    return i;
}
```

Листинг 6: torus_index

Функция next_state

Вход: `int row, int col` - координаты клетки

Выход: `int` - новое состояние клетки (0 или 1)

Описание: находит индексы верхнего, нижнего, левого и правого соседей через `torus_index`, затем формирует строку-ключ из пяти символов в порядке «верх, лево, центр, право, низ» (тот же порядок, что при построении таблицы в `Neumann_class`) и возвращает значение `neumann.neumann.at(key)`.

```
1  int MainWindow::next_state(int row, int col){
2      int up = torus_index(row - 1);
3      int down = torus_index(row + 1);
4      int left = torus_index(col - 1);
5      int right = torus_index(col + 1);
6
7      std::string key;
8      key += std::to_string(cells[up][col]);
9      key += std::to_string(cells[row][left]);
10     key += std::to_string(cells[row][col]);
11     key += std::to_string(cells[row][right]);
12     key += std::to_string(cells[down][col]);
13
14     return neumann.neumann.at(key);
15 }
```

Листинг 7: next_state

Функция make_step

Вход: текущее поле `cells`

Выход: поле на следующей итерации (записывается в `cells`)

Описание: для каждой клетки вызывает `next_state` и записывает результат в отдельную матрицу `next`. Только после обхода всех клеток `cells` заменяется на `next`; это обеспечивает синхронное обновление, при котором новые значения не влияют на вычисление соседних клеток. Затем увеличивается счетчик итераций и поле передается виджету.

```
1  void MainWindow::make_step(){
2      std::vector<std::vector<int>> next(field_size, std::vector<int
       >(field_size, 0));
3      for (int r = 0; r < field_size; ++r){
4          for (int c = 0; c < field_size; ++c){
5              next[r][c] = next_state(r, c);
6          }
7      }
8
9      cells = next;
10     iteration = iteration + 1;
11     grid_widget->set_grid(cells);
12 }
```

Листинг 8: make_step

Функция on_toggle_run

Вход: число итераций из поля ввода, состояние таймера `timer`

Выход: запущенный или остановленный таймер

Описание: слот кнопки «Старт» / «Стоп». Если таймер работает, он останавливается. Иначе число итераций считывается из поля ввода, записывается в счетчик `remaining_auto_steps` и запускается таймер с интервалом 150 мс.

```
1  void MainWindow::on_toggle_run(){
2      if (timer->isActive()){
3          timer->stop();
4          run_button->setText("Старт");
5      } else {
6          max_iterations = iters_spin->value();
7          remaining_auto_steps = max_iterations;
8          timer->start();
9          run_button->setText("Стоп");
10     }
11 }
```

Листинг 9: on_toggle_run

Функция on_auto_tick

Вход: счетчик `remaining_auto_steps`

Выход: выполненный шаг автомата либо остановка таймера

Описание: слот срабатывания таймера. Если оставшихся итераций нет, таймер останавливается. Иначе вызывается `make_step` и счетчик уменьшается на единицу, поэтому всего выполняется ровно заданное число итераций.

```
1  void MainWindow::on_auto_tick(){
2      if (remaining_auto_steps <= 0){
3          timer->stop();
4          run_button->setText("Старт");
5          return;
6      }
7
8      make_step();
9      remaining_auto_steps = remaining_auto_steps - 1;
10 }
```

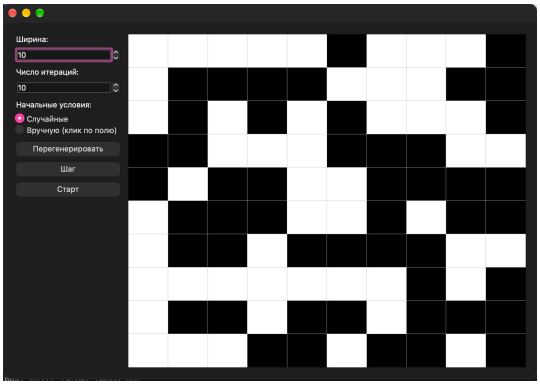
Листинг 10: on_auto_tick

Кнопка «Шаг» напрямую соединена со слотом `make_step` и выполняет одну итерацию.

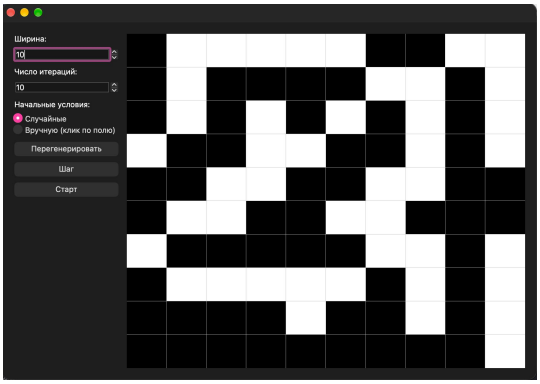
4 Результаты работы

4.1 Работа программы на случайных начальных условиях

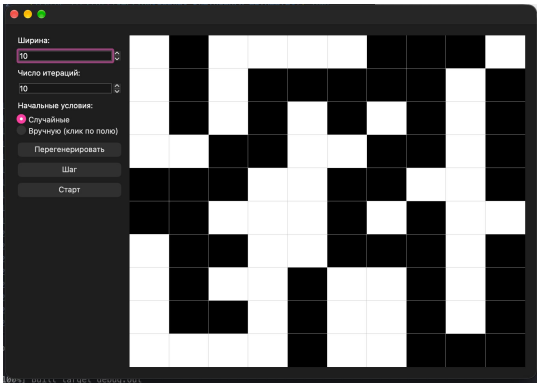
На рис.2 показаны начальное состояние и первые три итерации работы программы на случайном поле 10×10. Из начального «шума» за несколько шагов формируются регулярные фрагменты, похожие на диагональные полосы.



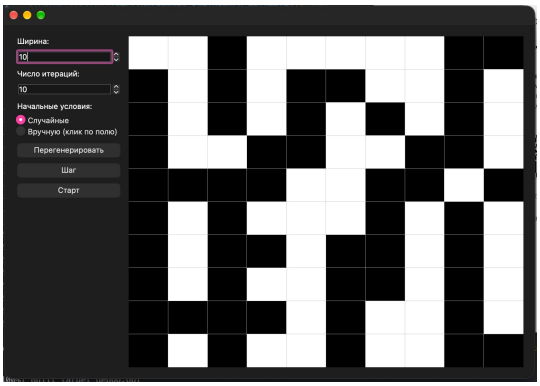
а) $t = 0$ (начальное состояние)



б) $t = 1$



в) $t = 2$

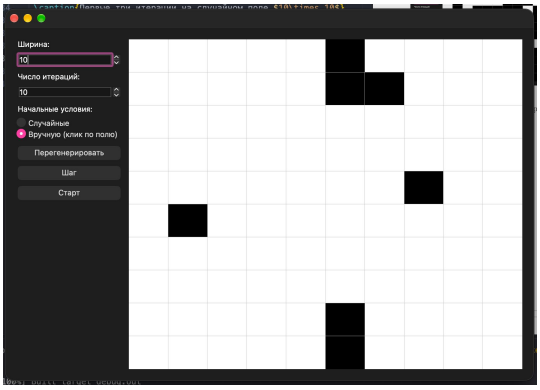


г) $t = 3$

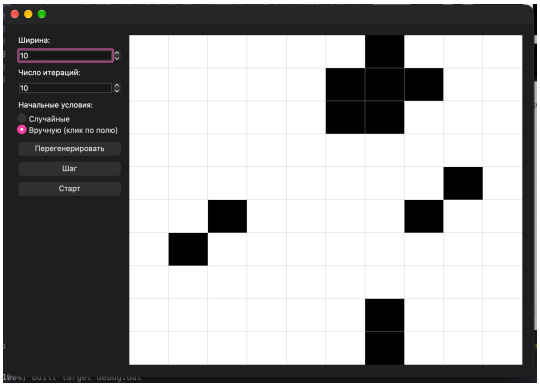
Рис. 2: Первые три итерации на случайном поле 10×10

4.2 Работа программы на ручных начальных условиях

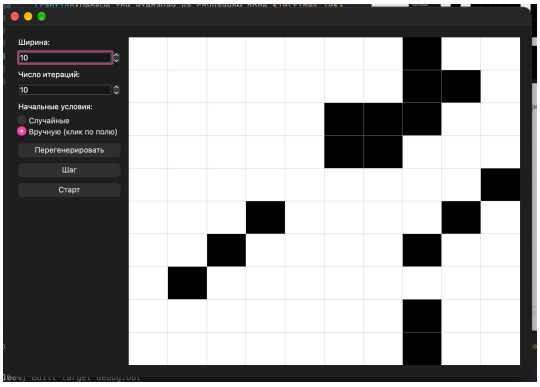
На рис.3 показаны начальное состояние, заданное вручную кликами по полю 10×10, и первые пять итераций.



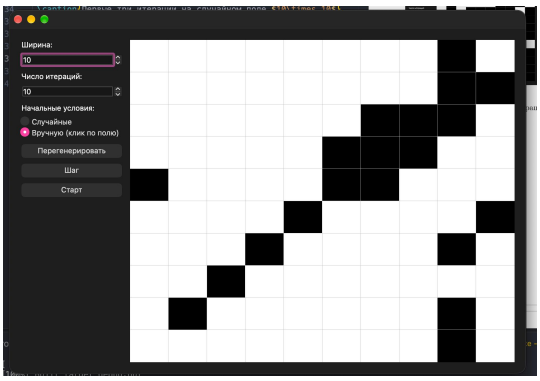
а) $t = 0$ (начальное состояние)



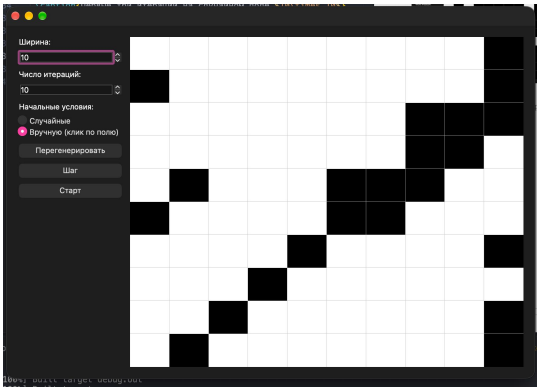
б) $t = 1$



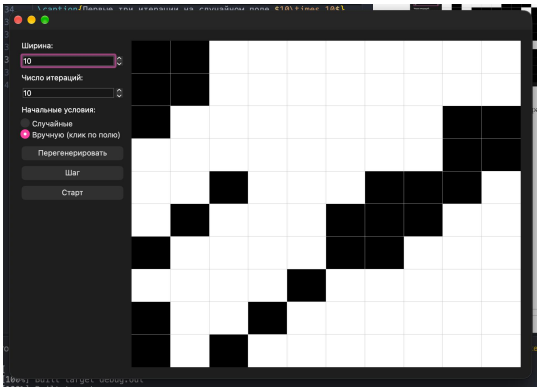
в) $t = 2$



г) $t = 3$



д) $t = 4$



е) $t = 5$

Рис. 3: Первые пять итераций при ручных начальных условиях, поле 10×10

5 Анализ клеточного автомата

5.1 Паттерны

5.1.1 Тривиальные начальные условия

Для начала рассмотрим два тривиальных начальных условия, когда все клетки или одновременно живы, или мертвы.

Пустое поле неподвижно, так как $f(0,0,0,0,0) = 0$, а полностью заполненное поле за один шаг становится пустым, так как $f(1,1,1,1,1) = 0$.

При нетривиальных начальных условиях можно выделить два вида поведения: шахматные доски, которые на каждой итерации инвертируют клетки, и диагональные волны, которые дрейфуют по полю. Рассмотрим их отдельно.

5.1.2 Шахматные доски

Существуют две шахматные доски: с живой и с мертвой клеткой в левом верхнем углу. В любой из них все четыре соседа каждой клетки имеют цвет, противоположный цвету самой клетки, поэтому используются только две строки таблицы истинности:

- живая клетка с мертвыми соседями, $f(0,0,1,0,0) = 0$ $(i = 4)$: клетка умирает
- мертвая клетка с живыми соседями, $f(1,1,0,1,1) = 1$ $(i = 27)$: клетка оживает

Следовательно, на каждом шаге все клетки меняют цвет, $x_{t+1} = 1 - x_t$: одна доска заменяется другой, а через два шага возвращается в исходное состояние, то есть пара шахматных досок образует цикл периода 2 шага (рис.4).

Однако цвета чередуются в шахматном порядке только при чётной ширине поля из-за тороидальных граничных условий. При нечетной ширине поля, после нескольких итераций поле прийдет к паттерну дрейфующих волн, описанному ниже.

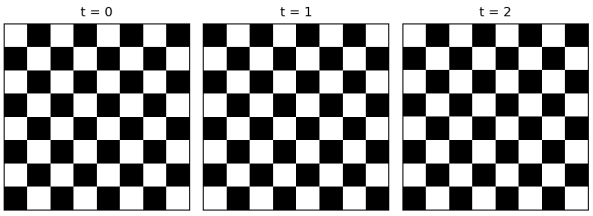


Рис. 4: Шахматное поле 8×8: цикл периода 2

5.1.3 Диагональные волны (дрейф вправо)

В отличие от шахматных досок, здесь закрашенные клетки не стоят на месте, а смещаются вправо, закрашивая соседа по диагонали. Наиболее чистый

пример даёт одиночная живая клетка (рис.5).

Таким образом, на каждом шаге живая клетка исчезает, но передаёт жизнь соседу справа и соседу снизу. Эти две новые клетки лежат на одной диагонали, поэтому из одной клетки возникает диагональный отрезок.

Волна на торе ширины n замыкается при $t = n - 1$, диагональ при этом состоит из n клеток.

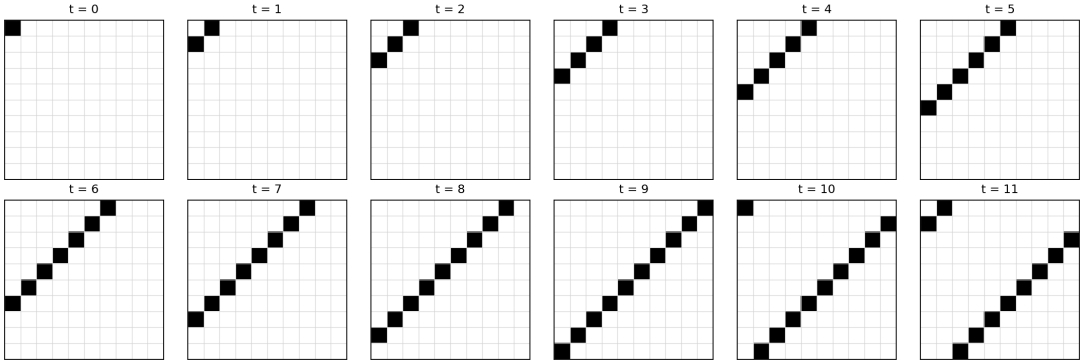


Рис. 5: Диагональная волна на торе 10×10

5.2 Классификация по Вольфраму

По наблюдаемому поведению автомат следует отнести к классу II, так как из случайного начального состояния он за короткое время переходит к набору периодических структур. Однако данная классификация носит оценочный характер, так как строгих критериев классификации Вольфрама не существует.

Заключение

В ходе выполнения лабораторной работы реализован двумерный клеточный автомат с окрестностью фон Неймана и тороидальными границами по варианту $N = 50045688$.

Реализовано:

1. Правило перехода в соответствии с номером варианта (таблица истинности булевой функции 5 переменных)
2. Поле с тороидальными граничными условиями
3. Возможность изменения начальных условий: ширины поля, числа итераций, типа генерации, режима работы (автоматический или пошаговый)

Достоинства реализованного решения:

1. Правило перехода строится из исходного числа автоматически, поэтому для другого варианта достаточно изменить одну константу.
2. Пользователю предоставлено два режима работы программы: с заданным числом итераций и пошаговый.
3. Таблица истинности строится один раз в конструкторе и хранится в ассоциативном массиве из 32 элементов, поэтому при вычислении шага для клетки нужен лишь поиск по ключу (не более $\log_2 32 = 5$ сравнений), а не вычисление значения булевой функции.

Недостатки и возможности для улучшения:

1. Номер текущей итерации хранится в программе, но не отображается в интерфейсе, также нет вывода статистики (например, доли живых клеток). Для этого можно реализовать отдельный виджет со статистикой.
2. Кнопка «Перегенерировать» не останавливает таймер. Если нажать её во время автоматического режима, оставшиеся итерации продолжат выполняться уже на новом поле. Решение - останавливать таймер и сбрасывать счетчик оставшихся шагов в `on_init`

Масштабируемость решения:

1. Добавить возможность сохранять и загружать файлы с состоянием поля.
2. Добавить поддержку правил, основанных не только на окрестности фон Неймана, но и на окрестности Мура.
3. Добавить возможность делать шаг не только вперед $(t + 1)$, но и назад $(t - 1)$.

Список литературы

[1] Лекции ВШТИИ по теории алгоритмов [Электронный ресурс]. URL: https://tema.spbstu.ru/algorithm/ (дата обращения: 04.10.2026)

[2] Модели клеточных автоматов - Викиконспекты ИТМО [Электронный ресурс]. URL: https://neerc.ifmo.ru/wiki (дата обращения: 04.10.2026).

[3] Wolfram, Stephen, A New Kind of Science. Wolfram Media, Inc., May 14, 2002.